Project 1 - The Citadel System

OS-P1-G15

Jan Francesc Porcar Guillen - janfrancesc.porcar

Francesc Mateu Vallespín - f.mateu

Operating Systems – Year 25/26– The Citadel System– Version 1.1

Table of Contents

Introduction....1
System Overview flow....2
Module Overview....3
Data Structures & Justification....4
Helper Functions....8
Command loop....9
Signal Handling....10
Network Architecture & Socket Design....14
Trade and Product Requests....17
I/O Constraints....19
Network Communication....21
Pledge Protocol (Alliance System)....21
Intermediary Hop (Routing)....23
Product List and Trade Orders....23
Concurrency and Shared State....24
MD5 Integrity Verification....24
Envoy Creation....29
Mission Assignment Syste....31
Sending a Mission to an Envoy....33
Envoy Execution Loop....34
Mission Result Handling....35
Envoy Status Command....37
External Command Classification....39
Trade Order Delegation....41
Graceful Shutdown with Envoys....43
Synchronization of Shared State....44

Introduction

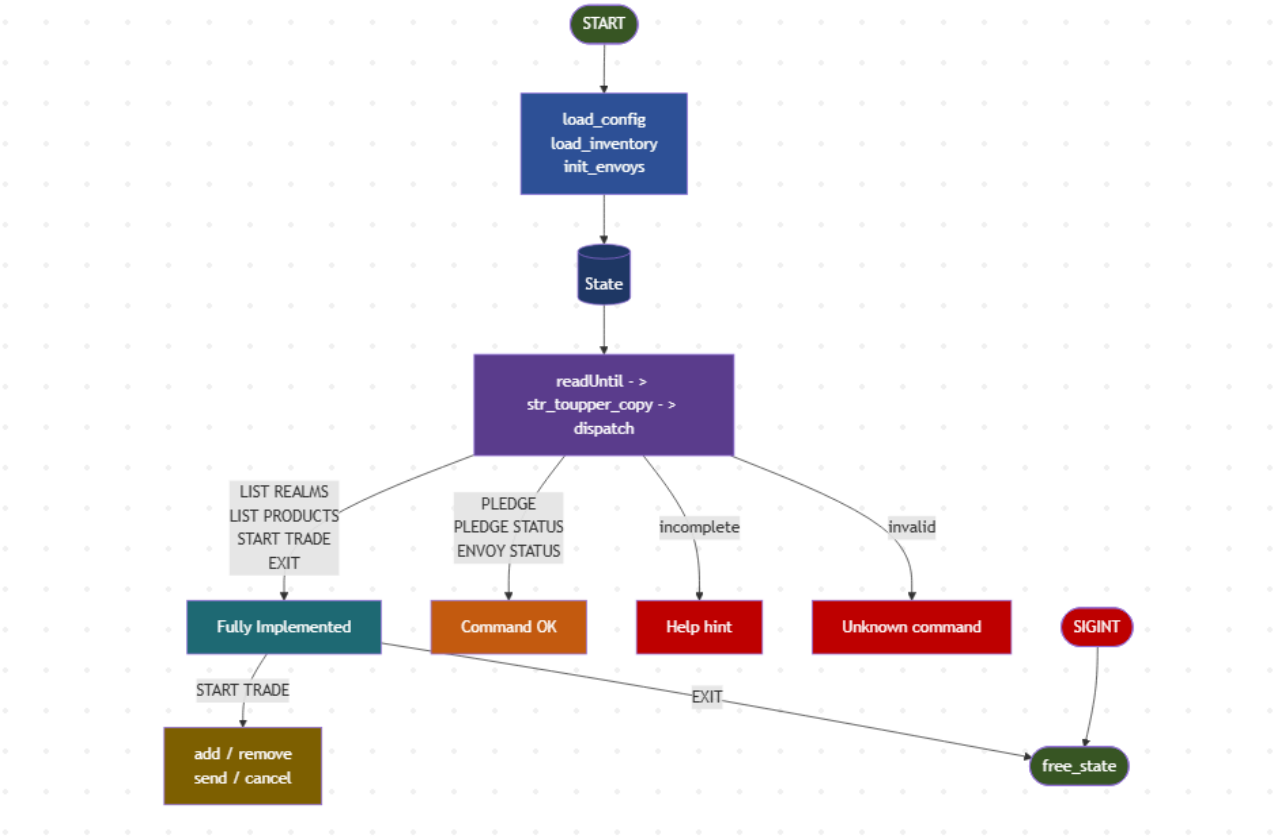
Citadel System is a decentralized network simulation where independent processes, each simulating a Maester for a Noble House, communicate and form alliances and engage in trade within the realm. Each Maester reads its configuration and inventory from local files and has its own command terminal for the user's interaction.

Phase 1 lays the groundwork for the architecture of the system, including initializing processes, configuration, binary management, and an interactive command terminal for the user, which is robust and has case-insensitive input handling.

Phase 2 introduces TCP socket-based networking between Maesters. Each process now listens on its configured IP and port, accepts incoming connections from other Maesters, and is capable of sending and receiving structured binary frames over the network. This phase implements the alliance (PLEDGE) handshake, product listing requests, and trade order submission.

This document will outline the chosen data structures and rationale, the breakdown of the modules into seven translation units, the processes, and the decisions made throughout the implementation.

System Overview flow



In Phase 1 the Maester is a single process. There is no networking yet — the architecture is designed as a clean base so that Phase 2 (sockets), Phase 3 (file transfer), and Phase 4 (Envoys via fork) can be added incrementally without restructuring existing code.

In Phase 2, each Maester becomes a networked node. At startup, a background listener thread is launched that accepts incoming TCP connections on the port specified in maester.dat. For each connection, a new detached thread is spawned to handle the incoming frame. The main thread continues running the interactive command loop concurrently. The two threads synchronize shared state — alliances and pending pledges — via a pthread_mutex_t embedded in the Maesterstruct. When a frame's destination does not match the local realm, it is forwarded to the next hop according to the routing table, enabling multi-hop communication between Maesters without direct routes.

Module Overview

The project is split into seven translation units, each with a single clear responsibility.

File	Responsibility	Key Functions
main.c	Entry point, init, cleanup	main(), free_state()
config.c	Parse maester.dat	load_config(), free_config()
inventory.c	Binary stock.db I/O and display	load_inventory(), display_inventory()
commands.c	Command loop and dispatcher	run_command_loop(), dispatch()
trade.c	START TRADE interactive sub-loop	start_trade_session()
utils.c	I/O and string helpers	readUntil(), str_trim(), str_toupper_copy()
signals.c	SIGINT/SIGTERM handling	setup_signals(), signal_handler()
alliance.c	Alliance table init	init_alliances(), free_alliances()
envoy.c	Envoy array init	load_envoys(), free_envoys()
network2.c	Socket listener, frame routing, alliance/trade networking	start_listener(), stop_listener(), send_pledge_request(), send_pledge_response(), send_trade_order(), notify_allied_realms_exit(), check_pending_timeouts()
protocol2.c	Socket listener, frame routing, alliance/trade networking	serialize_frame(), parse_frame(), compute_frame_checksum(), set_frame_data()

Data Structures & Justification

4.1 Route

```
typedef struct {
    char *name;
    char *ip;
    int   port;
    int   known;
} Route;
```

One entry in the routing table. All strings are dynamic (char *) because the names and IPs come from parsing the config file and their lengths are not known at compile time. The known flag distinguishes routes with a real IP from wildcard routes (*.*.*.*)

4.2 Config

```
typedef struct {
    char  *realm;
    char  *path;
    int    num_envoys;
    char  *ip;
    int    port;
    Route *routes;
    int    num_routes;
    Route  default_route;
    int    has_default;
} Config;
```

All string fields are char * because they come from parsing maester.dat with strdup. The routes array is dynamic because the number of realms in the file is not known until parsing is complete — it grows with realloc one entry at a time.

4.3 Product

```
typedef struct {
   char   name[100];    ->     fixed - matches binary file spec
exactly
    int   amount;
    float weight;
} Product;
```

The only fixed-size array in the project. The project specification defines the binary format of stock.db as exactly char name[100], int amount, float weight. The struct layout must match the file layout so that read() can load a record in one call with no serialization.

4.4 Envoy

```
typedef enum { FREE, ON_MISSION } EnvoyStatus;

typedef struct {
    int id;
    EnvoyStatus status;
} Envoy;
```

Envoys are allocated as a fixed array because num_envoys is known from the config before the command loop starts. In Phase 4 this struct will gain a pid_t field for the real child process.
In Phase 2 the struct gains a mission_realm (char *) field recording which realm the envoy was dispatched to, displayed by ENVOY STATUS

4.5 Alliance

```
typedef enum { PENDING, ON_PENDING, ALLIED, FAILED, REJECTED
} AllianceStatus;

typedef struct {
    char *realm;
    AllianceStatus status;
} Alliance;
```

The alliance table starts empty (NULL) and grows only when PLEDGE commands are processed in Phase 2. In Phase 1 all PLEDGE commands are stubs that print Command OK.
In Phase 2, the Alliance struct gains ip and port fields so that once an alliance is established the Maester can connect directly to the allied realm without consulting the routing table.

4.6 Maester

```
typedef struct {
    Config    config;
    Inventory inventory;
    Envoy    *envoys;
    int       num_envoys;
    Alliance *alliances;
    int       num_alliances;
} Maester;
```

Central container passed by pointer to every module. Grouping all runtime data here avoids global variables (except keep_running which must be global for signal safety) and makes cleanup trivial: one free_maester() call releases everything.

4.7 Pending Pledge

```
typedef struct {
    unsigned long request_id;
    char *realm;
    char *ip;
    int port;
    int envoy_id;
    PendingType type;
    time_t created_at;
} PendingPledge;
```

Tracks in-flight alliance handshakes. The type field (PENDING_OUTGOING / PENDING_INCOMING) distinguishes whether the local Maester initiated the request or received it. The created_at timestamp is used by check_pending_timeouts() to expire pledges that have not been answered within 120 seconds, at which point the envoy is released and the alliance is marked FAILED. All string fields are char * because realm names and IPs are parsed at runtime from configuration files and incoming frames.

4.8 Frame

```
typedef struct {
    uint8_t       type;
    char          origin[20];
    char          destination[20];
    uint16_t      data_length;
    unsigned char data[275];
    uint16_t      checksum;
} Frame;
```

Fixed-size binary frame used for all inter-Maester communication. The fixed layout of exactly 320 bytes matches the protocol specification in Annex II and allows serialization and deserialization with simple memcpy operations, guaranteeing a constant wire size for every message regardless of content. The 16-bit checksum is computed as the sum of all bytes modulo 65536, providing basic integrity verification on every received frame. The origin and destinationfields use null-padded fixed arrays so that no dynamic allocation is needed during parsing. The data_length field is transmitted in network byte order using htons/ntohs.

Helper Functions

The three helper functions in utils.c are used throughout the codebase for I/O and string handling.

5.1 readUntil

```
char *readUntil(int fd, char separator);
```

Reads characters from a file descriptor one byte at a time using read() until the separator character is found. Returns a dynamically allocated string. Used for all input: reading maester.dat line by line, reading stock.db would not use it (binary), and reading all terminal commands from STDIN. The buffer grows with realloc as needed so there is no fixed limit on input length.

5.2 str_toupper_copy

```
void str_toupper_copy(char *dst, const char *src, int maxlen);
```

Copies src into dst converting every character to uppercase. Used in dispatch() to make command recognition case-insensitive. We copy into a separate buffer instead of modifying in place because we still need the original casing to extract arguments like realm names.

5.3 str_trim

```
void str_trim(char *s);
```

Removes leading and trailing whitespace in place using memmove. Needed because readUntil may return strings with trailing \r characters (Windows line endings) or extra spaces, which would cause strcmp to fail silently.

Command loop

The command loop executes three tasks which include reading user input and transforming the command into a standardized format and executing the appropriate system operation. The system exists as a continuous operation which maintains its execution until the Maester system terminates.

The loop reads commands from STDIN using readUntil(). This method maintains project integrity because it requires read() as the sole input method which excludes standard input functions like scanf and fgets.

The dispatcher manages the execution of each command. The dispatcher generates multiple command versions before it proceeds to evaluate the input.

```
char *upper = malloc(strlen(line) + 1);
char *ucopy = malloc(strlen(line) + 1);
char *orig  = malloc(strlen(line) + 1);
```

The upper version allows command comparison to functions in a case-insensitive manner. The ucopy version supports safe modifications through strtok while the orig version maintains the complete original text to prevent realm name and argument text from becoming uppercase.

The distinct separation between these two elements exists because commands require case-insensitive processing while realm names need to maintain their original appearance.

1. Read and clean the command
2. Compare it with the supported commands
3. Extract arguments if needed
4. Execute the command
5. Show an error or help message when needed

Signal Handling

The Maester program uses signal handling to enable a clean shutdown process which activates when users stop the program by pressing CTRL+C or when the system receives a termination signal.

The project uses sigaction() instead of signal() because it gives more predictable behavior and allows the program to configure how signals are handled.

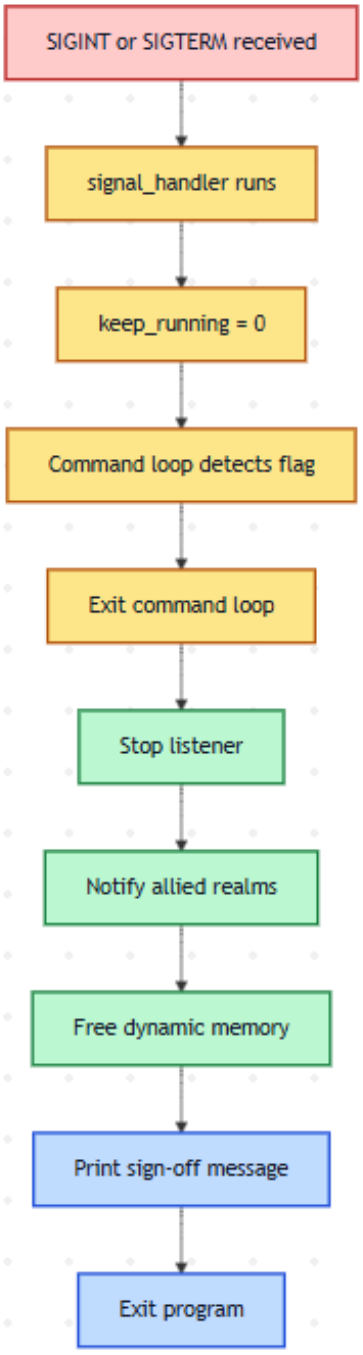
The signal handler shows its design as a basic program which changes only the global keep_running flag.

```c
volatile sig_atomic_t keep_running = 1;

void signal_handler(int signum) {
    (void)signum;
    keep_running = 0;
}
```

The handler does not call malloc free printf or other complex functions This is important because signal handlers must only use async-signal-safe operations The real cleanup operation does not occur inside the handler process

The command loop detects that keep_running has changed and exits the program in a normal way. This design keeps the program safe The Maester program can stop the listener which, notifies allied realms if needed, and frees dynamic memory after the loop has finished its execution.



Protocol Design & Pledge Handshake Flow

The second phase establishes network links between Maesters through the use of TCP sockets. The binary frame structure which exists in protocol2.c defines the format which all messages need to use when they travel between realms.

The protocol aims to achieve consistent network operations throughout its entire network environment. The same frame format handles the transmission of pledge requests, pledge responses, product list requests, trade orders, acknowledgements, errors, and disconnect notifications.

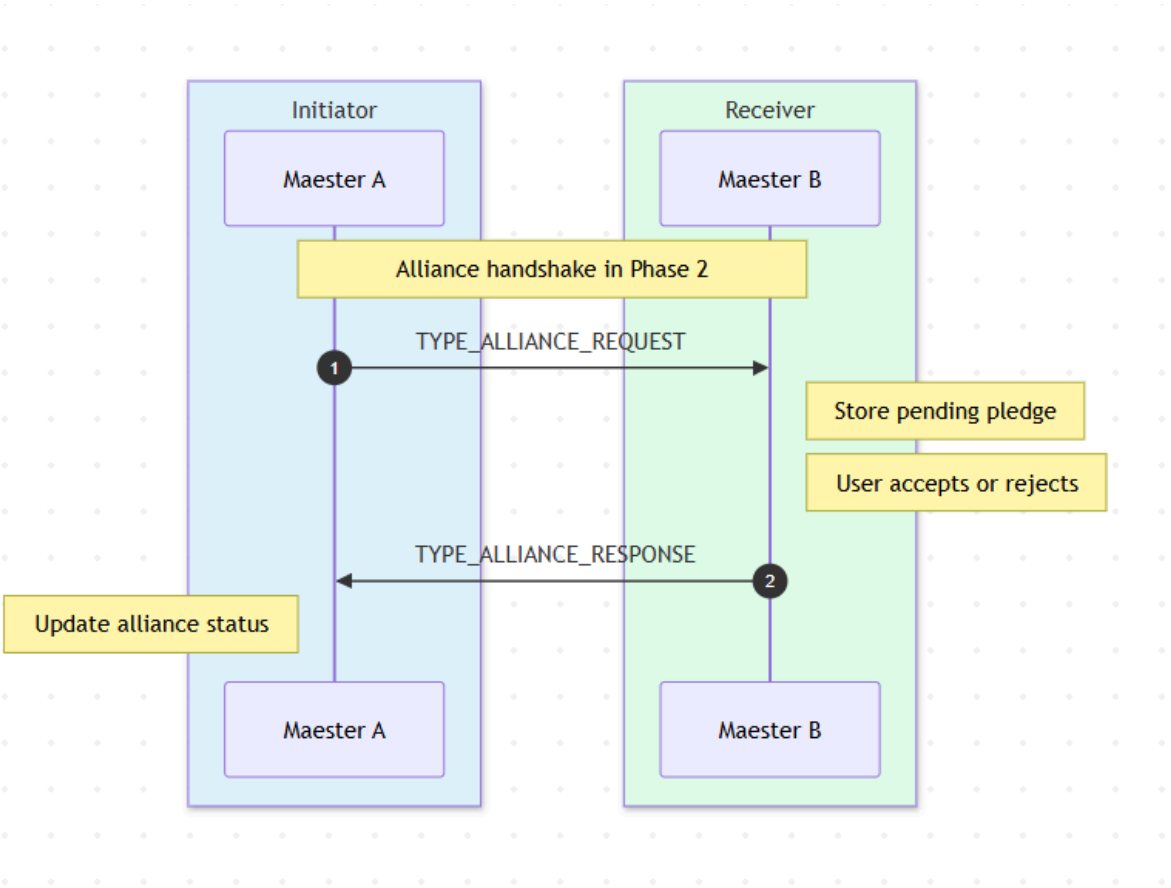
```
typedef struct {
uint8_t type;
char origin[20];
char destination[20];
uint16_t data_length;
unsigned char data[275];
uint16_t checksum;
} Frame;
```

The system designates each frame to maintain a specific frame dimension. This makes communication easier because the receiver always knows how many bytes must be read from the socket. The type field identifies the operation, origin stores the sender address, destination stores the target realm, data contains the message payload, and checksum verifies the frame content delivery. The program first serializes a frame before it proceeds to send the frame after it generates a checksum. The system first parses a frame before it proceeds to validate the frame. The frame gets rejected when the checksum shows an error.

TYPE_ALLIANCE_REQUEST	Sends a pledge request
TYPE_ALLIANCE_RESPONSE	Accepts or rejects a pledge
TYPE_PRODUCT_LIST_REQUEST	Requests products from another realm
TYPE_ORDER_REQUEST	Sends a trade order
TYPE_DISCONNECT	Notifies that a Maester is existing
TYPE_ACK / TYPE_NACK	Confirms success or failure

The pledge handshake works in two steps. First, one Maester sends an alliance request to another realm. The receiver stores the request as pending and waits for the user to accept or reject it. The receiver sends back an alliance response.

Allied status will become available to both Maesters when they accept the pledge. The alliance will not form if the pact receives rejection or expiration because the envoy will be permitted to leave.



Maesters turn into network nodes during Phase 2. The program execution begins when it creates a listener thread which establishes a TCP socket connection through the IP address and port number specified in maester.dat.

The main thread keeps running the command loop, while the listener thread waits for incoming connections. The user can continue entering commands while the Maester processes messages from other realms.

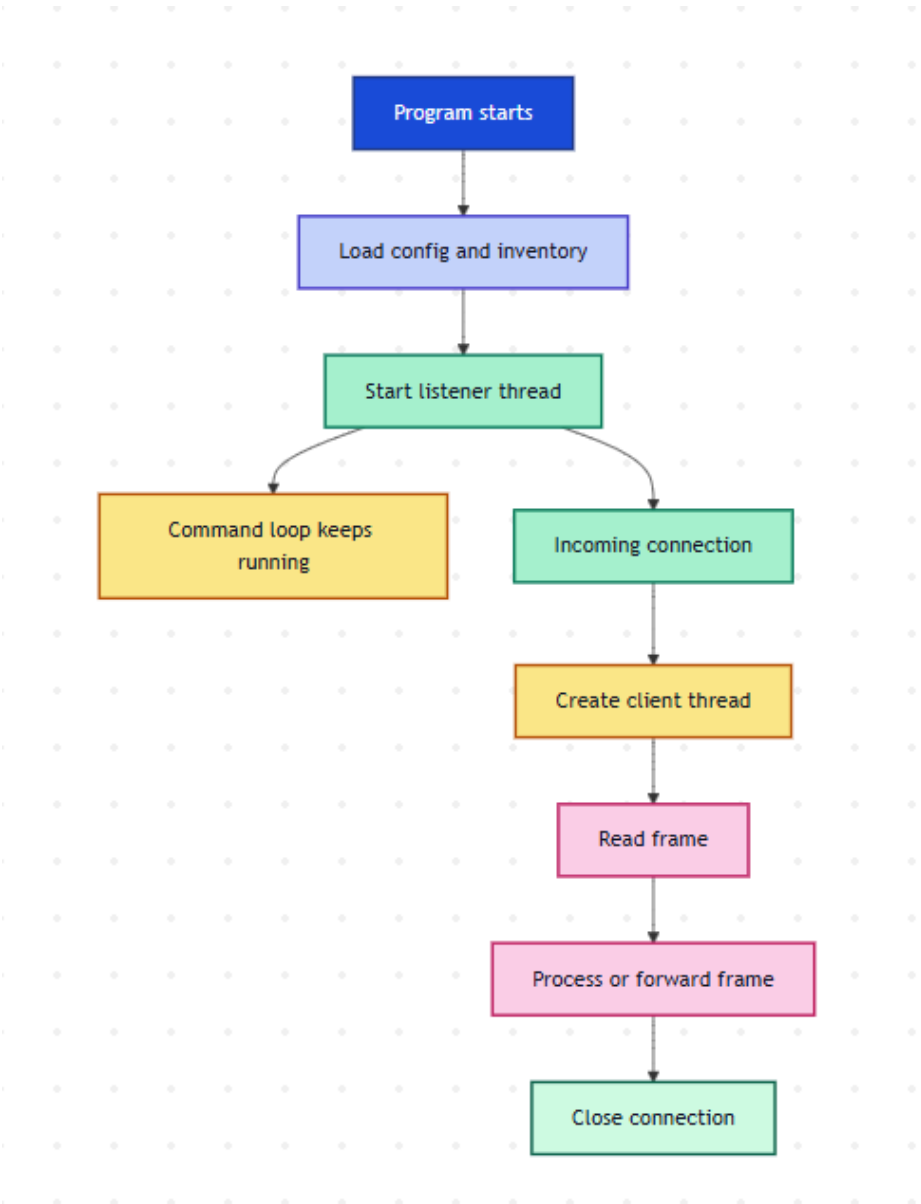
The listener establishes a separate client thread which handles new incoming connections. This client thread reads one frame, processes it, sends a response if needed, and then closes the connection.

Network Architecture & Socket Design

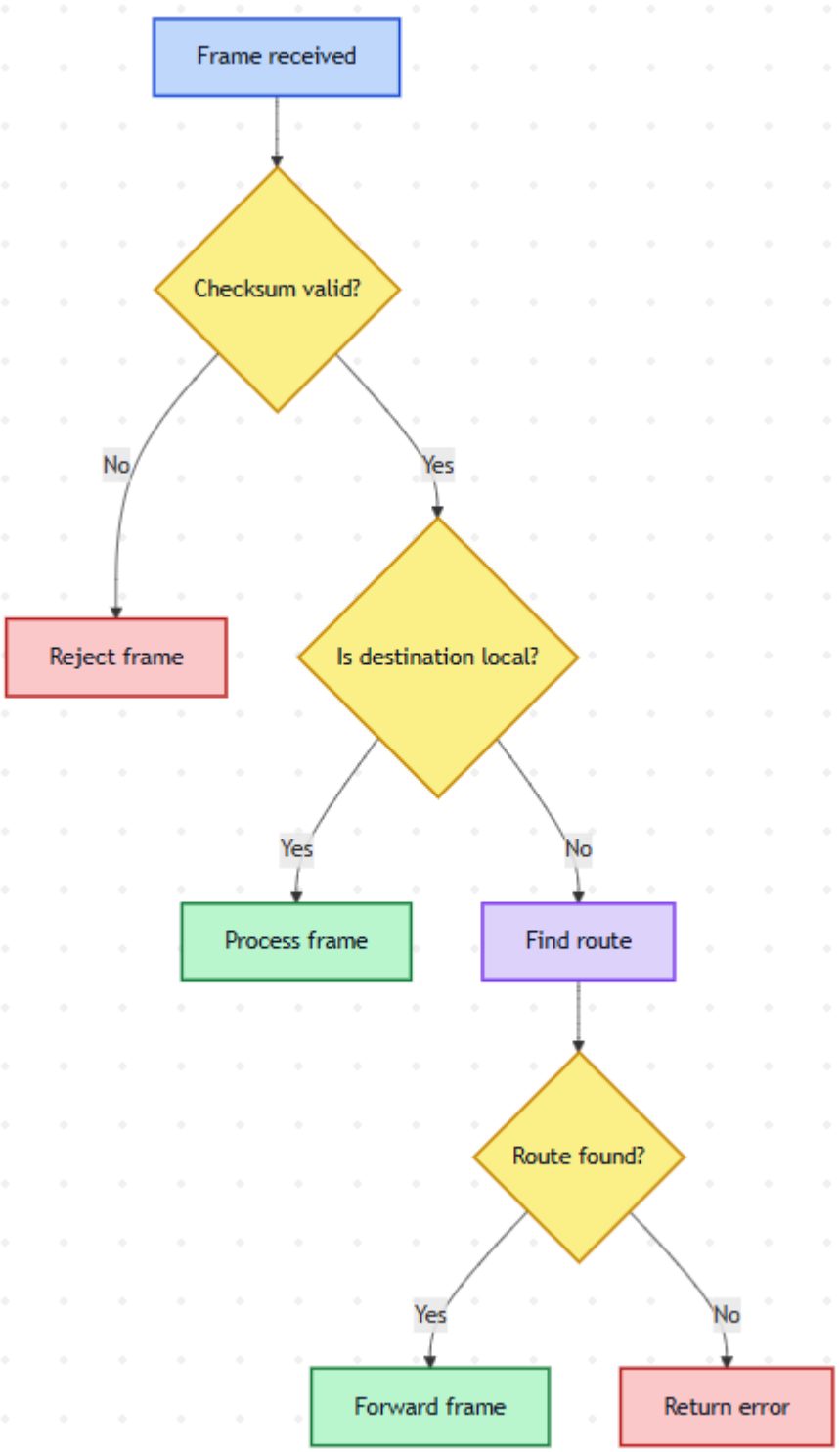
The system initiates a TCP socket connection through maester.dat which defines its IP address and port.

The listener thread functions to establish incoming connections while the main thread continues executing commands. The user can input commands while the Maester simultaneously receives messages from different realms.

The listener establishes a separate client thread for every new connection. The client thread processes one frame after reading it and sends a response when necessary before ending the connection.



The frame processing logic is simple. The Maester checks the destination of the received frame to determine its processing method. The frame is processed locally when the destination matches the Maester's realm. The Maester uses the routing table to find the next hop which it sends to after receiving a frame.



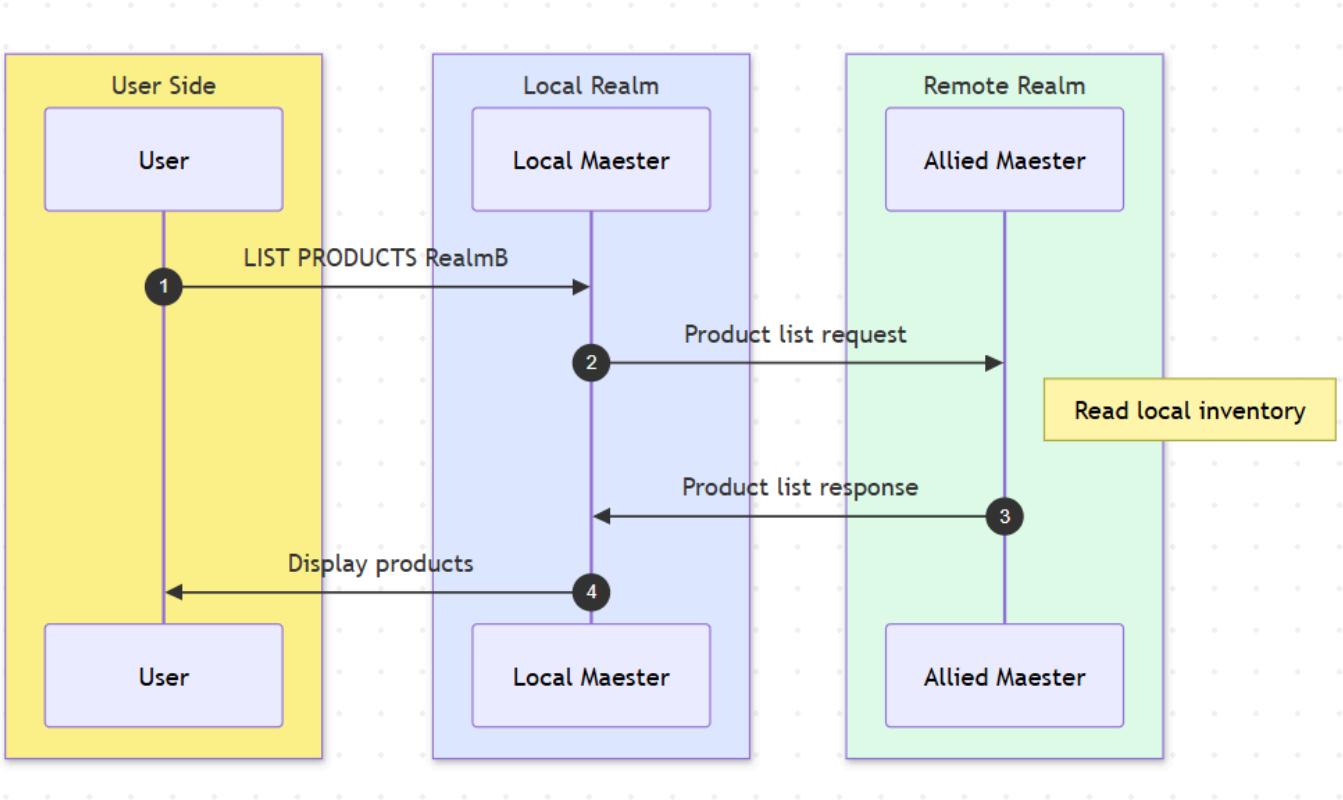
The routing system enables realms to communicate without needing direct contact. A Maester does not need a direct connection to every other realm. The system requires only knowledge of the next hop to operate successfully.

Multiple threads need to access the same information so shared structures require mutex protection. The system needs this protection to maintain control over alliances and pending pledges and envoys and request identifiers.

```
pthread_mutex_lock(&maester->lock);
pthread_mutex_unlock(&maester->lock);
```

Trade and Product Requests

The Maester checks whether the target realm is allied. If it is, it sends a product list request frame. The receiving Maester reads its local inventory and sends back the product information.

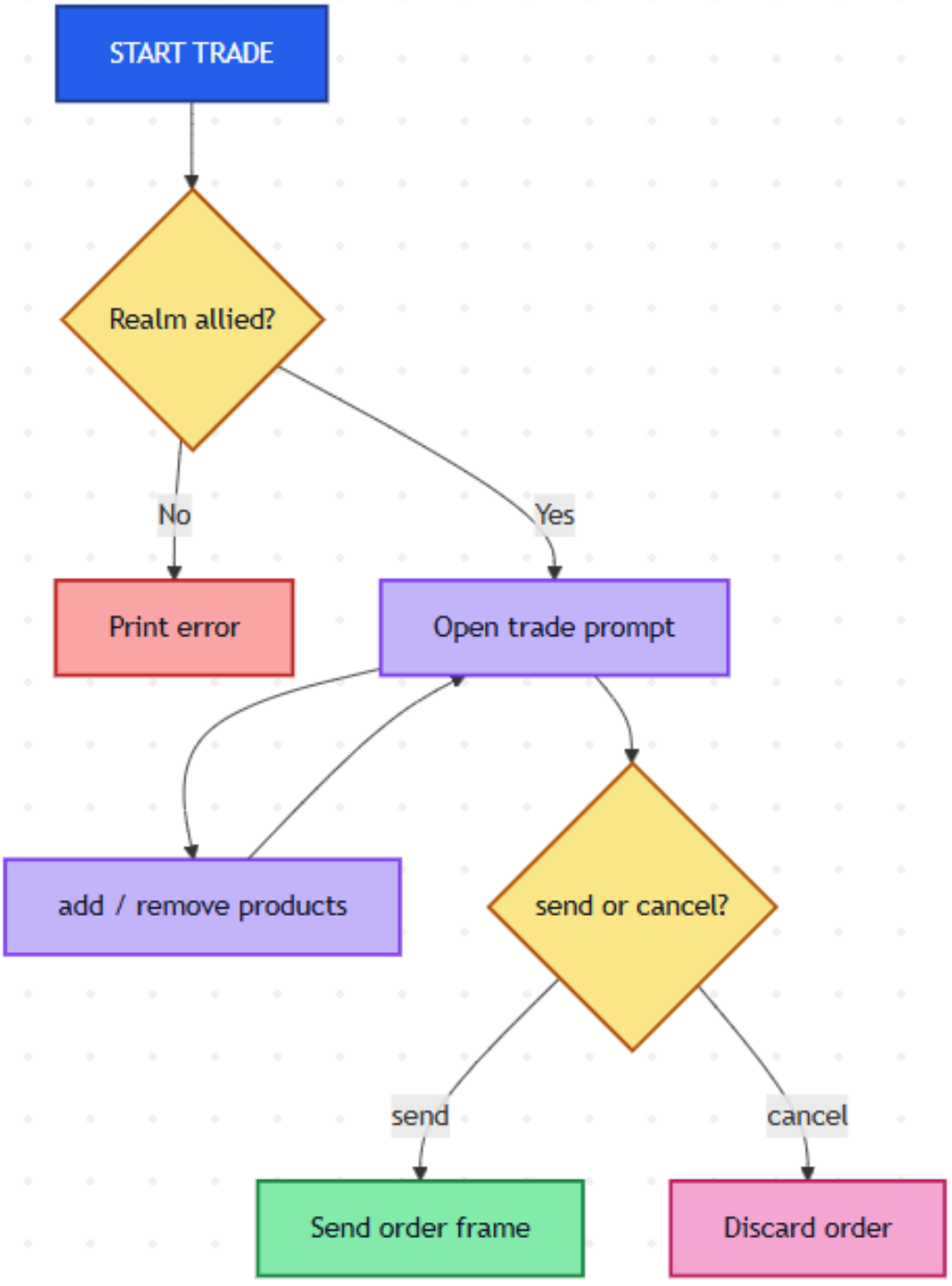


START TRADE

The interface enables users to create new product entries while they can also delete existing product entries and proceed with their order submission process or terminate their active session.

add <product> <amount>	Adds or updates a product in the order
remove <product> <amount>	Reduces or removes a product
send	Sends the order to the allied realm
cancel	Cancels the trade session

The Maester constructs the order payload when the user places an order and transmits it as a network frame to the allied realm.


START TRADE
Realm allied?
No
Yes
Print error
Open trade prompt
add / remove products
send or cancel?
send
cancel
Send order frame
Discard order

I/O Constraints

The project meets its required I/O restrictions because it does not use standard C input and output functions which include printf and scanf and fgets and fprintf and puts. The implementation uses UNIX system calls which enable low-level operations through its primary functions read() and write().

The system handles all user interactions through its command line while its internal processes use file handling methods. The system uses system calls together with their built-in functions to manage all user input and configuration parsing and terminal output and binary file access.

The project uses the printF macro to control all terminal output through its central printing function.

```
#define printF(x) write(STDOUT_FILENO, (x), strlen(x))
```

The macro maintains code readability while it requires direct usage of write() function. The modules share a helper which enables them to print messages instead of needing to write write(STDOUT_FILENO, ...) multiple times.

The program generates a complete formatted string which it stores in memory before sending it to printF for output. The program immediately frees the memory space after displaying the output.

```
char *msg = NULL;

asprintf(&msg, "Maester of %s initialized.\n",
maester.config.realm);

if (msg) {
    printF(msg);
    free(msg);
}
```

The system operates with a message pattern which displays runtime values including realm names and ports and product names and envoy status and alliance status and network errors. The system produces output through an adaptable system which does not depend on printf.

The program uses its custom readUntil() helper to handle input operations. The function reads from a file descriptor one byte at a time until it finds a separator which usually appears as the character \n.

```
char *line = readUntil(STDIN_FILENO, '\n');
```

The same helper function can handle various types of text input which include reading terminal commands and parsing lines from maester.dat. The program can handle any command size because it uses a dynamically allocated string as its return value.

Binary data is handled differently. The inventory file stock.db is read using read() directly into Product structures, because the binary format is fixed by the assignment. The process eliminates the need for extra parsing while maintaining the file structure according to its memory representation.

```
Product p;

while (read(fd, &p, sizeof(Product)) == sizeof(Product)) {
    /* process product */
}
```

Phase 2 also uses the same low-level I/O model for sockets. Network frames are sent and received through file descriptors, keeping terminal, file, and network operations consistent with the UNIX read() / write() approach.

Terminal Output	write() through the printF macro
Terminal Input	read() through the readUntil() helper
Bynary files and Sockets	Direct system calls with fixed-size structures

Network Communication

Each Maester listens on a TCP port configured in its `.dat` file. A dedicated listener thread calls `accept()` in a loop and spawns a new detached thread per incoming connection. Each connection thread reads exactly one frame, processes it, and closes the socket.

We chose TCP over UDP because ordered reliable delivery is required for chunked file transfers: data chunks must arrive in order and without loss. The fixed 320-byte frame format (1 type + 20 origin + 20 destination + 2 data_length + 275 data + 2 checksum) means one `read()` always delivers exactly one complete frame, with no need for reassembly logic. The checksum (sum of all bytes mod 65536) detects corruption and triggers a `NACK` reply.

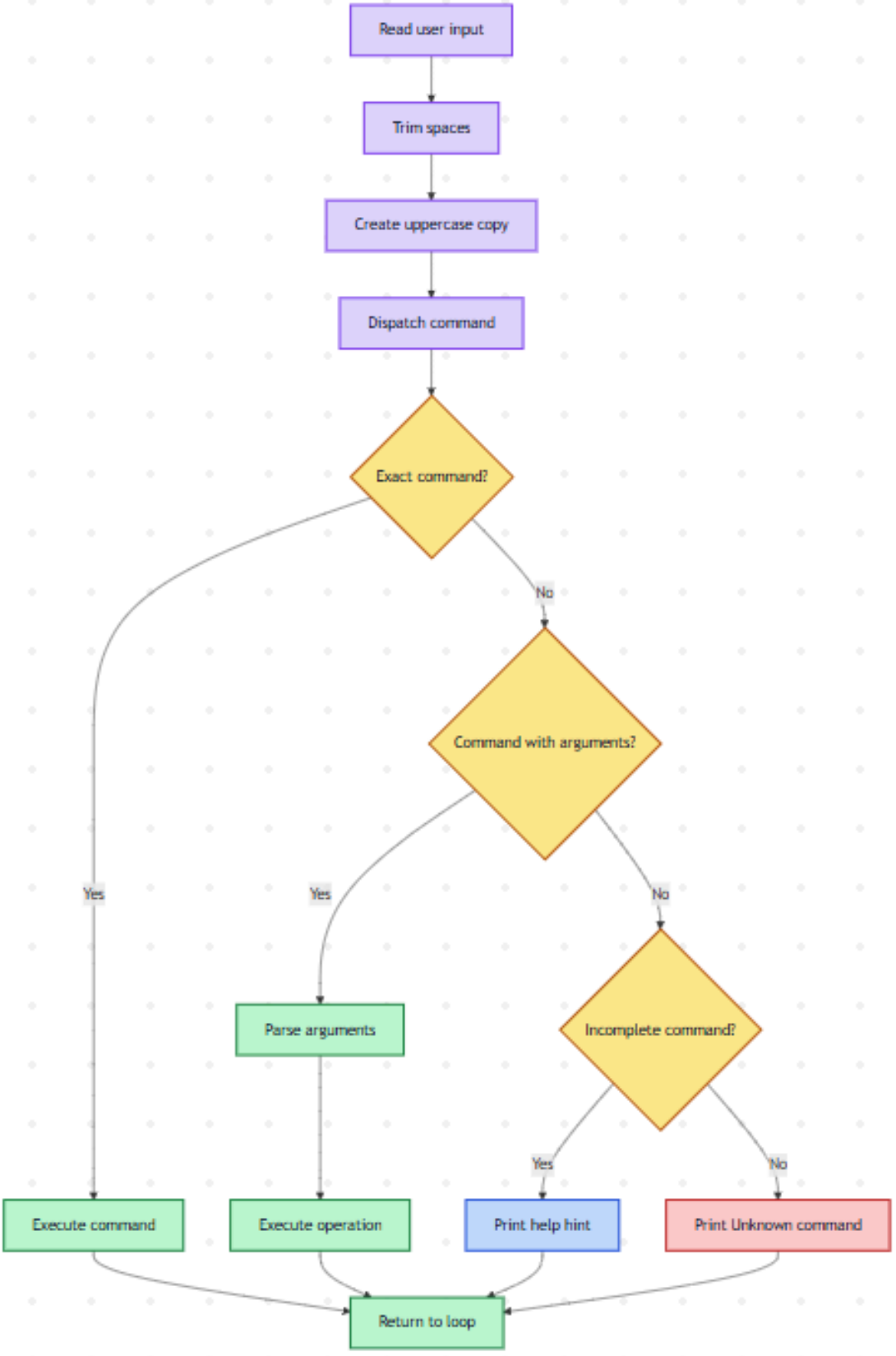
Pledge Protocol (Alliance System)

A realm initiates an alliance by sending `PLEDGE <Realm> <Sigil>`. The sigil file is read into memory, its MD5 is computed, and the payload is sent as a chunked transfer (`TYPE_ALLIANCE_REQUEST` header + data chunks + `ACK_MD5`), all on a single persistent TCP connection.

The receiving realm stores the incoming pledge in the `pending_pledges` array and notifies its user. The user can then run `PLEDGE RESPOND <Realm> ACCEPT|REJECT`, which sends a `TYPE_ALLIANCE_RESPONSE` frame back.

Once accepted, the `alliances` array is updated to status `ALLIED` with the peer's IP and port. From that point, all traffic to that realm goes direct without passing through intermediaries.

Outgoing pledges have a 120-second timeout, checked at every iteration of the main REPL loop (which uses `select()` with a 1-second timeout on stdin). If no response arrives in time, the status is set to `FAILED` and the user is notified.


Read user input
Trim spaces
Create uppercase copy
Dispatch command
Exact command?
No
Command with arguments?
Yes
Yes
No
Parse arguments
Incomplete command?
Yes
No
Execute command
Execute operation
Print help hint
Print Unknown command
Return to loop

Intermediary Hop (Routing)

If a realm has no direct route to a destination, it forwards through an intermediary configured in its routing table. The config file lists named routes and an optional `DEFAULT` fallback. `find_route()` searches named entries first, then falls back to `DEFAULT`.

The key challenge was that the intermediary must proxy the full transfer — not just the header. We solved this with `proxy_chunked_transfer()`: after forwarding the header and receiving `ACK_FILE`, it enters a `select()` relay loop that reads data chunks from the sender and forwards them to the destination, until the final `ACK_MD5` arrives and is relayed back to the sender. This works correctly over any number of hops.

Product List and Trade Orders

`LIST PRODUCTS <Realm>` sends a `TYPE_PRODUCT_LIST_REQUEST` to an allied realm. The receiver serialises its inventory as `name|amount|weight` lines and sends it back as a chunked transfer. The requester stores the result in memory (needed before `START TRADE` is allowed).

`START TRADE <Realm>` opens an interactive sub-REPL where the user builds an order with `ADD`, `REMOVE`, `LIST`, `SEND`, and `CANCEL`. On `SEND`, the order is serialised as `item:qty,item:qty,...` and sent as a chunked transfer (`TYPE_ORDER_REQUEST_HEADER` + data).

The receiving realm checks stock for every item. If all quantities are available it deducts them, saves the inventory to disk, and sends `TYPE_ORDER_RESPONSE ACCEPTED`. Otherwise it replies `REJECTED` without touching the stock. The requester updates its own inventory on `ACCEPTED`. Inventory is stored as a flat binary file (direct `read()`/`write()` of `Product` structs) so it persists across restarts.

Concurrency and Shared State

The main thread runs the REPL loop. A second thread (`listener_thread`) runs `accept()` and spawns a detached thread per connection. All shared Maester state (alliances, pending pledges, inventory, transfer metadata) is protected by a single `pthread_mutex_t`.

The REPL loop uses `select()` with a 1-second timeout instead of blocking on `readUntil()`. This allows periodic timeout checks without an extra thread and ensures clean exit when `SIGINT` sets `keep_running = 0` (`select()` returns -1 on `EINTR`).

Asynchronous notifications (alliance forged, order accepted, timeout expired) are printed by listener threads with a `\n` prefix so they push past any already-printed `$ ` prompt, and a `$ ` suffix to give the user a fresh prompt after the interruption.

MD5 Integrity Verification

Every chunked transfer includes an MD5 hash of the payload, computed by the sender before transmitting. The receiver reassembles all chunks in a heap buffer, recomputes the MD5, and compares. On match it replies `ACK_MD5`; on mismatch it replies `NACK`.

MD5 is computed via `popen("md5sum <tmpfile>")`: the payload is written to a secure temp file created with `mkstemp()`, the `md5sum` command is run as a subprocess, and the 32-character hex result is read back. The temp file is deleted with `unlink()` immediately after. This avoids any crypto library dependency and uses only POSIX primitives.

Envoy Process Architecture

The system now has real Envoy processes. Each Envoy gets made by the Maester using fork() during startup, after reading some configuration file, or well I guess, the file that tells it what to do. The number of Envoys comes from maester.dat, so every realm can set its own concurrency capacity, in its own kinda way.

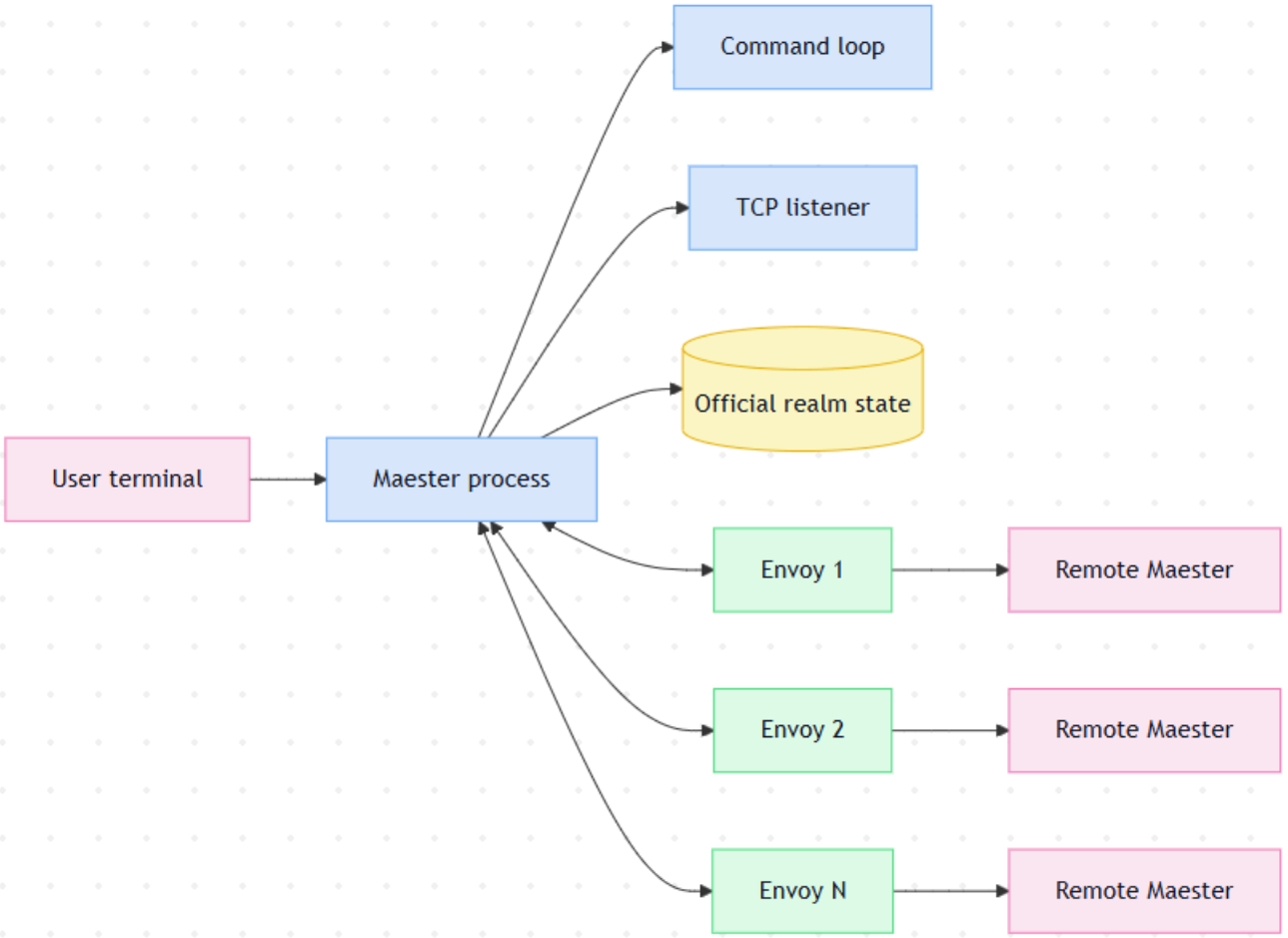
The Maester still acts like the realm coordinator. It keeps the command loop, the TCP listener, the routing logic, and the “official” state. The Envoys are worker processes that run outgoing missions like PLEDGE, LIST PRODUCTS and START TRADE , so the Maester isn’t stuck doing everything.

```
typedef enum {
    FREE,
    ON_MISSION
} EnvoyStatus;

typedef enum {
    MISSION_NONE,
    MISSION_PLEDGE,
    MISSION_LIST_PRODUCTS,
    MISSION_TRADE
} MissionType;

typedef struct {
    int id;
    pid_t pid;
    EnvoyStatus status;
    MissionType mission_type;
    char *mission_realm;
    int to_envoy_fd;
    int from_envoy_fd;
} Envoy;
```

The pid field shows the real child process id. The status field is what ENVOY STATUS looks at, and mission_type plus mission_realm describe what the Envoy is currently handling, right then. The pipe file descriptors are used for internal talk with the Maester, kind of a direct channel, nothing fancy.


Command loop
TCP listener
Official realm state
User terminal
Maester process
Envoy 1
Remote Maester
Envoy 2
Remote Maester
Envoy N
Remote Maester

Maester and Envoy Communication

The Maester communicates with each Envoy through pipes. Two pipes per Envoy, one pipe for pushing missions from the Maester to the Envoy, and another pipe for shipping results back, from the Envoy to the Maester. No sockets are used for that internal back and forth. Sockets are kept reserved for communication between different Maesters, because you know, separation matters.

```
typedef struct {
    MissionType type;
    char realm[100];
    char file_path[256];
    char order_path[256];
} MissionMessage;

typedef struct {
    int envoy_id;
    MissionType type;
    int success;
    char realm[100];
    char message[256];
} EnvoyResult;
```

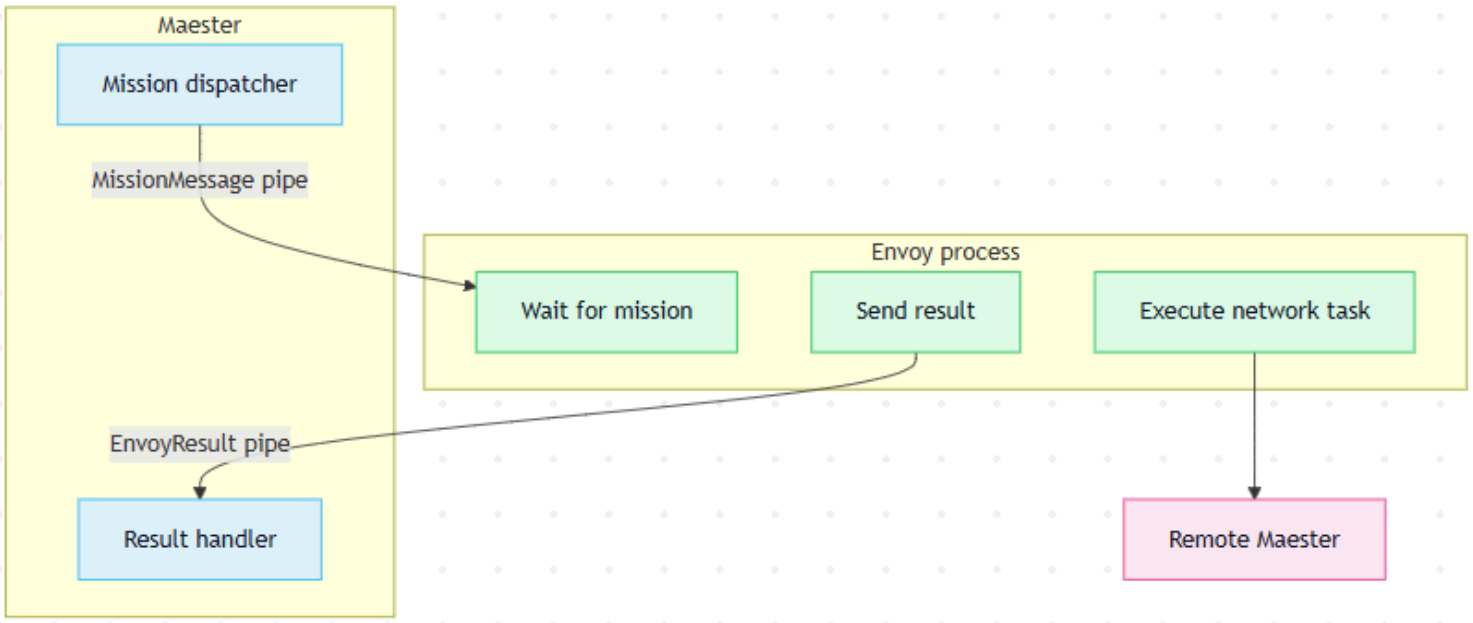
MissionMessage holds what an Envoy needs to execute a single mission. EnvoyResult is returned when the mission finishes. This design means the Envoy doesn't get direct ownership of the Maester state, like, it can do the work but not touch the core records too freely.

```
if (pipe(to_envoy) < 0) { return (-1);}

if (pipe(from_envoy) < 0) {
close(to_envoy[0]);
close(to_envoy[1]);
return (-1);
}

pid = fork();
```

The pipes get created before fork(), so when fork happens both the Maester and the Envoy inherit the descriptors. After fork each process closes the pipe ends it does not use, otherwise things get muddled, or so they say.



Envoy Creation

Envoys are started during Maester initialization. The parent saves the child PID and the pipe descriptors. The child drops into a loop waiting for missions.

```
pid = fork();

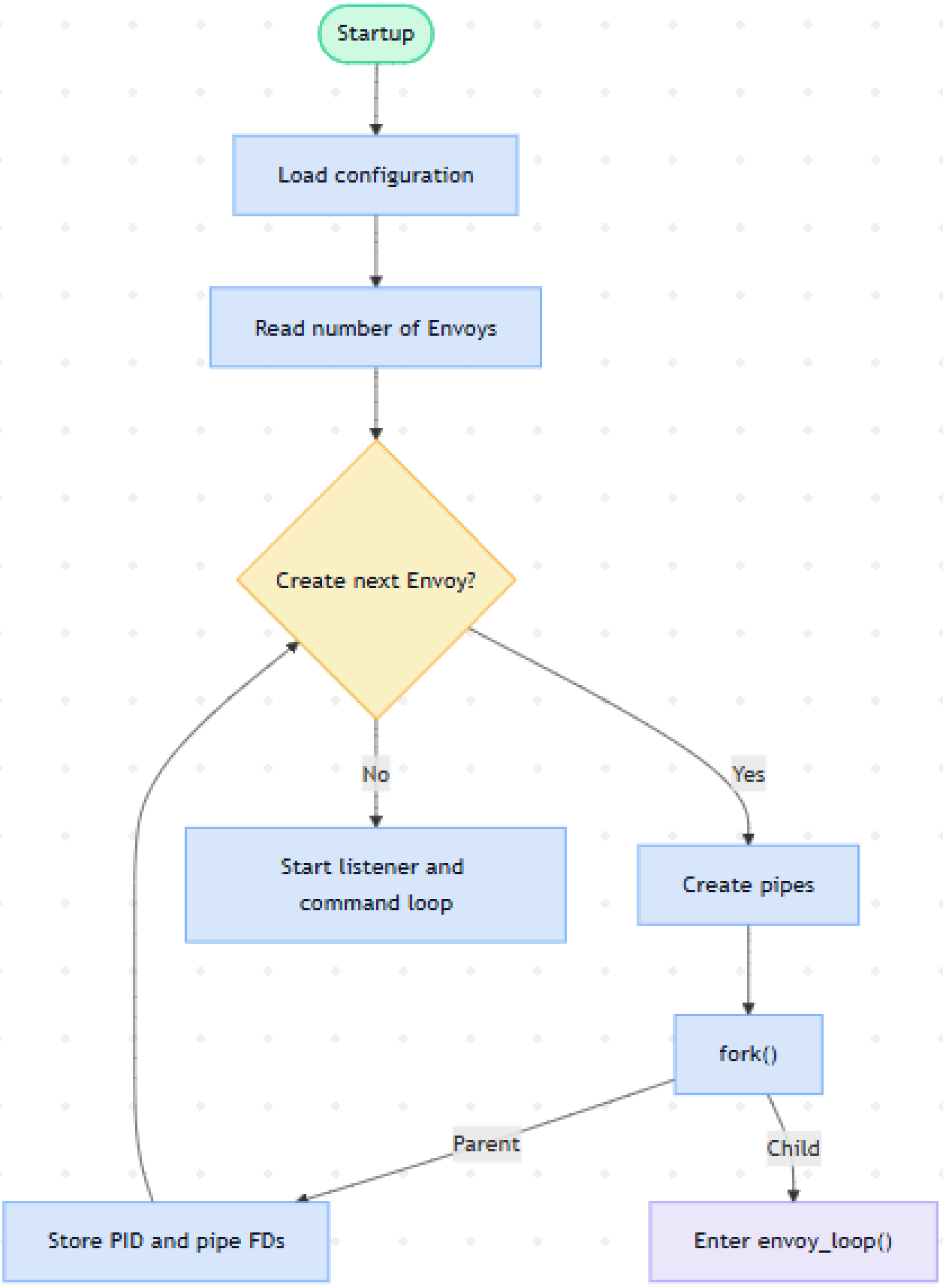
if (pid < 0) {
    return (-1);
}

if (pid == 0) {
    close(to_envoy[1]);
    close(from_envoy[0]);
    envoy_loop(to_envoy[0], from_envoy[1], envoy_id);
    exit(0);
}

close(to_envoy[0]);
close(from_envoy[1]);

envoys[i].id = envoy_id;
envoys[i].pid = pid;
envoys[i].status = FREE;
envoys[i].mission_type = MISSION_NONE;
envoys[i].to_envoy_fd = to_envoy[1];
envoys[i].from_envoy_fd = from_envoy[0];
```

In that child loop, it closes the write end of the mission pipe and closes the read end of the result pipe. The parent does the reverse. That way each direction stays clear, and nobody ends up reading from the wrong side, accidentally.


Startup
Load configuration
Read number of Envoys
Create next Envoy?
No
Yes
Start listener and command loop
Create pipes
fork()
Parent
Child
Store PID and pipe FDs
Enter envoy_loop()

Mission Assignment System

External commands need a free Envoy, or well, that's what it boils down to. Before the command dispatcher even starts a mission, it checks whether an Envoy is sitting around and available. If none shows up, the mission just doesn't get going, full stop.

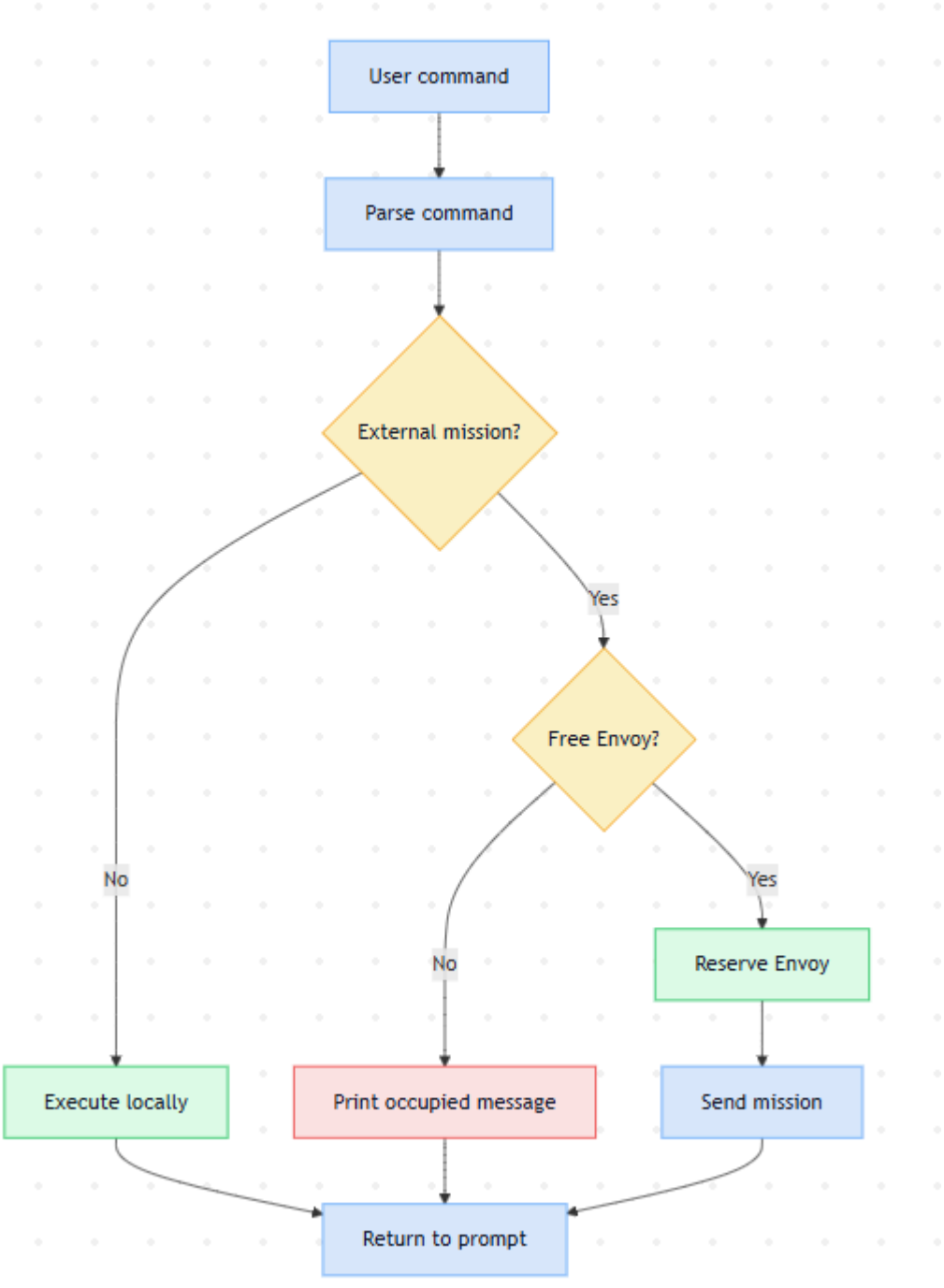
```
int acquire_free_envoy(Maester *maester, MissionType type,
const char *realm) {
    int i;

    pthread_mutex_lock(&maester->lock);
    i = 0;
    while (i < maester->num_envoys) {
        if (maester->envoys[i].status == FREE) {
            maester->envoys[i].status = ON_MISSION;
            maester->envoys[i].mission_type = type;
            free(maester->envoys[i].mission_realm);
            maester->envoys[i].mission_realm = strdup(realm);
            pthread_mutex_unlock(&maester->lock);
            return (i);
        }
        i++;
    }
    pthread_mutex_unlock(&maester->lock);
    return (-1);
}
```

This function also guards the Envoy table using the Maester mutex. As soon as an Envoy is reserved, nobody else can grab it, until the reservation gets released. Like, that's the whole point in the simplest sense, control.

```
if (envoy_id < 0) {
    printF("All envoys are occupied. Your command must
wait.\n");
    return;
}
```

It’s basically for concurrency management. The system can launch several missions simultaneously, but it must never run more missions than the configured Envoys can support. So the limit is not “soft”, it’s hard capped by how many Envoys exist.



Sending a Mission to an Envoy

After an Envoy has been reserved, the Maester builds a MissionMessage and then writes it into the Envoy pipe. The write happens as a fixed-size structure.

```
int assign_envoy_mission(Maester *maester, int envoy_id,
MissionMessage *mission) {
    int fd;

    if (envoy_id < 0 || envoy_id >= maester->num_envoys) {
        return (-1);
    }

    fd = maester->envoys[envoy_id].to_envoy_fd;
    if (write(fd, mission, sizeof(MissionMessage)) !=
sizeof(MissionMessage)) {
        return (-1);
    }

    return (0);
}
```

That's okay because it stays internal to the pipe, it's not part of the outside 320-byte Maester protocol. So, no external contract issues there.

```
MissionMessage mission;

memset(&mission, 0, sizeof(MissionMessage));
mission.type = MISSION_PLEDGE;
strncpy(mission.realm, realm, sizeof(mission.realm) - 1);
strncpy(mission.file_path, sigil_path,
sizeof(mission.file_path) - 1);

assign_envoy_mission(maester, envoy_id, &mission);
```

Envoy Execution Loop

On the Envoy side, there's a loop. Each Envoy waits for pipe data, and once it receives a MissionMessage, it inspects the mission type. Then it runs the matching network operation.

```
void envoy_loop(int read_fd, int write_fd, int envoy_id) {
    MissionMessage mission;
    EnvoyResult result;
    ssize_t n;

    while ((n = read(read_fd, &mission,
sizeof(MissionMessage))) > 0) {
        memset(&result, 0, sizeof(EnvoyResult));
        result.envoy_id = envoy_id;
        result.type = mission.type;
        strncpy(result.realm, mission.realm,
sizeof(result.realm) - 1);
        if (mission.type == MISSION_PLEDGE) {
            result.success = envoy_execute_pledge(&mission);
        } else if (mission.type == MISSION_LIST_PRODUCTS) {
            result.success =
envoy_execute_list_products(&mission);
        } else if (mission.type == MISSION_TRADE) {
            result.success = envoy_execute_trade(&mission);
        } else {
            result.success = 0;
        }

        write(write_fd, &result, sizeof(EnvoyResult));
    }

    close(read_fd);
    close(write_fd);
}
```

The Envoy doesn't directly touch the Maester state. It only hands back a result. That choice prevents multiple processes from changing the official state at the same time, which… tends to be messy and unpredictable.

Mission Result Handling

When the Maester gets the result from the Envoy, it updates the official state and releases the Envoy for reuse.

```
void handle_envoy_result(Maester *maester, EnvoyResult
*result) {
    pthread_mutex_lock(&maester->lock);

    if (result->success) {
        apply_success_result(maester, result);
    } else {
        apply_failure_result(maester, result);
    }

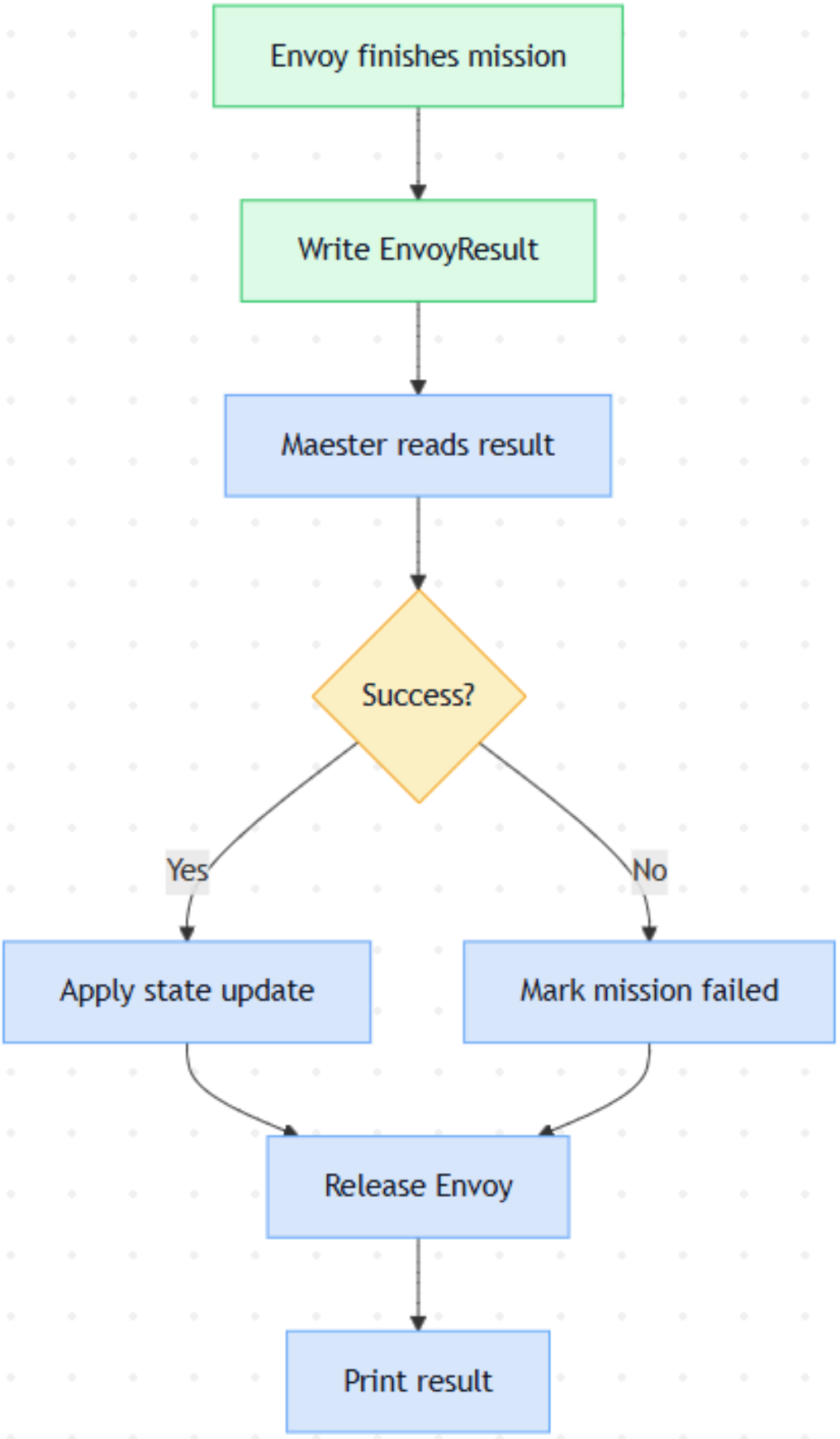
    release_envoy_locked(maester, result->envoy_id);

    pthread_mutex_unlock(&maester->lock);
}
```

That’s how ENVOY STATUS stays correct right after each mission ends.

```
void release_envoy_locked(Maester *maester, int envoy_id) {
    if (envoy_id < 0 || envoy_id >= maester->num_envoys) {
        return;
    }

    maester->envoys[envoy_id].status = FREE;
    maester->envoys[envoy_id].mission_type = MISSION_NONE;
    free(maester->envoys[envoy_id].mission_realm);
    maester->envoys[envoy_id].mission_realm = NULL;
}
```


Envoy finishes mission
Write EnvoyResult
Maester reads result
Success?
Yes
No
Apply state update
Mark mission failed
Release Envoy
Print result

Envoy Status Command

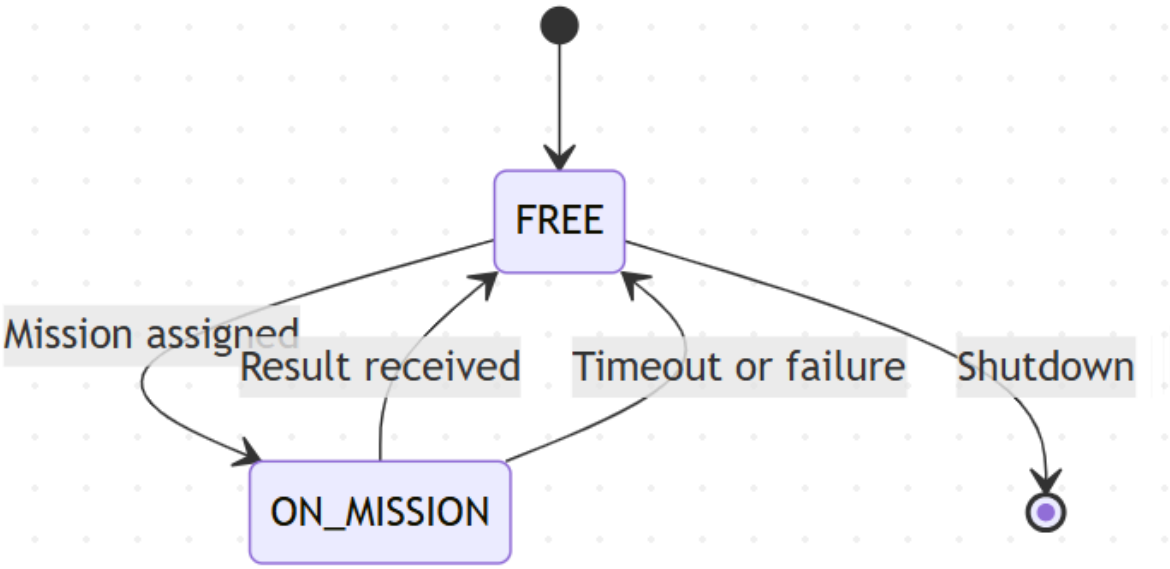
The ENVOY STATUS command is local, so it does not require an Envoy. It only reads local state and returns it, no reserved resource needed.

```
void print_envoy_status(Maester *maester) {
    int i;
    char *msg;

    pthread_mutex_lock(&maester->lock);
    i = 0;
    while (i < maester->num_envoys) {
        if (maester->envoys[i].status == FREE) {
            asprintf(&msg, "- Envoy %d: FREE\n",
maester->envoys[i].id);
        } else {
            asprintf(&msg, "- Envoy %d: ON_MISSION (%s to
%s)\n",
                     maester->envoys[i].id,

mission_type_to_string(maester->envoys[i].mission_type),
                     maester->envoys[i].mission_realm);
        }
        if (msg != NULL) {
            printF(msg);
            free(msg);
        }
        i++;
    }
    pthread_mutex_unlock(&maester->lock);
}
```

This is handy if you want to observe concurrency in real time. If two Envoys are busy, the user can see which missions are currently active, even if everything else keeps moving.


FREE
Mission assigned
Result received
Timeout or failure
Shutdown
ON_MISSION

External Command Classification

The command dispatcher splits “local” commands from the external mission kind of commands. This prevents Envoys from being assigned to things that do not really need network communication, and it also keeps the worker processes from getting dragged into operations that may sit around waiting for another Maester. So in practice, the Maester can execute those simple local commands right away, while anything outgoing is throttled by the Envoy availability system.

```
int command_requires_envoy(const char *upper_line) {
    if (strncmp(upper_line, "PLEDGE ", 7) == 0) {
        return (1);
    }
    if (strncmp(upper_line, "LIST PRODUCTS ", 14) == 0) {
        return (1);
    }
    if (strncmp(upper_line, "START TRADE ", 12) == 0) {
        return (1);
    }
    return (0);
}
```

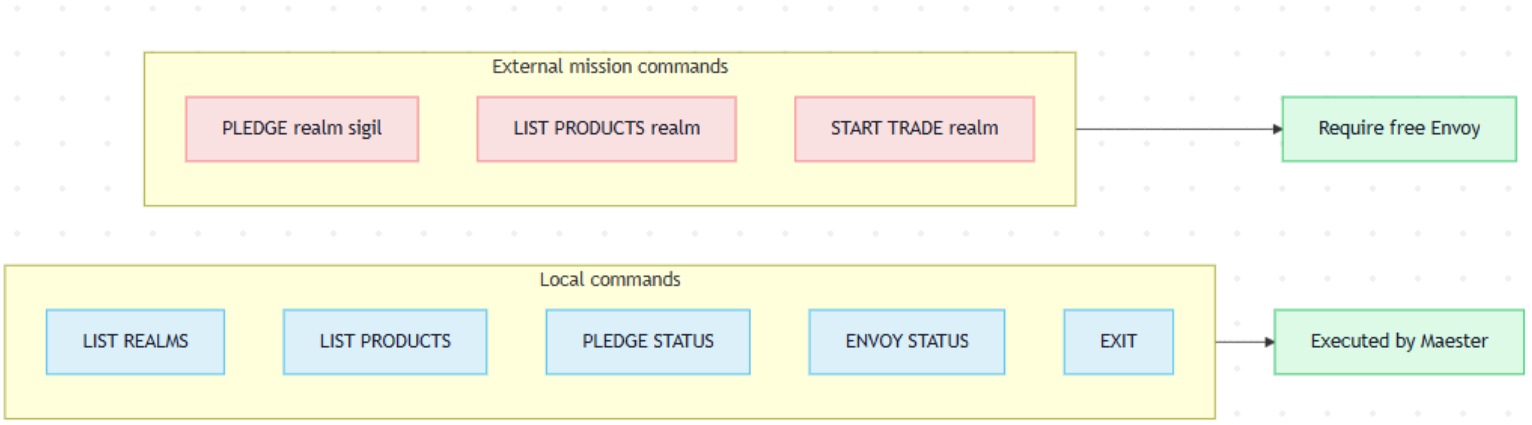
Local commands stay inside the Maester because they only check, or fine tune, information that’s already present in the same local process, nothing fancy, no extra realm needed.

```
LIST REALMS
LIST PRODUCTS
PLEDGE STATUS
ENVOY STATUS
EXIT
```

External mission commands are delegated because they require communication with another realm and so they can involve sockets, routing, file transfer, acknowledgements, or just plain waiting for a response from somewhere remote.

```
PLEDGE <REALM> <sigil>
LIST PRODUCTS <REALM>
START TRADE <REALM>
```

This split also makes the overall behavior easier to describe and easier to test. For example if a command wants to send ravens out of the local realm, it first has to obtain a free Envoy. When there isn't any Envoy available, then the mission just does not begin.



Trade Order Delegation

The trade menu remains interactive, but the real sending of the order is handed off to an Envoy. The reason is that creating the shopping list depends on user input and can be handled locally, while transmitting the final order needs network communication with another Maester. So the Maester controls the interactive part, and only the remote operation is delegated.

```
if (strcmp(command, "send") == 0) {
    envoy_id = acquire_free_envoy(maester, MISSION_TRADE,
realm);
    if (envoy_id < 0) {
        printF("All envoys are occupied. Your command must
wait.\n");
        return;
    }

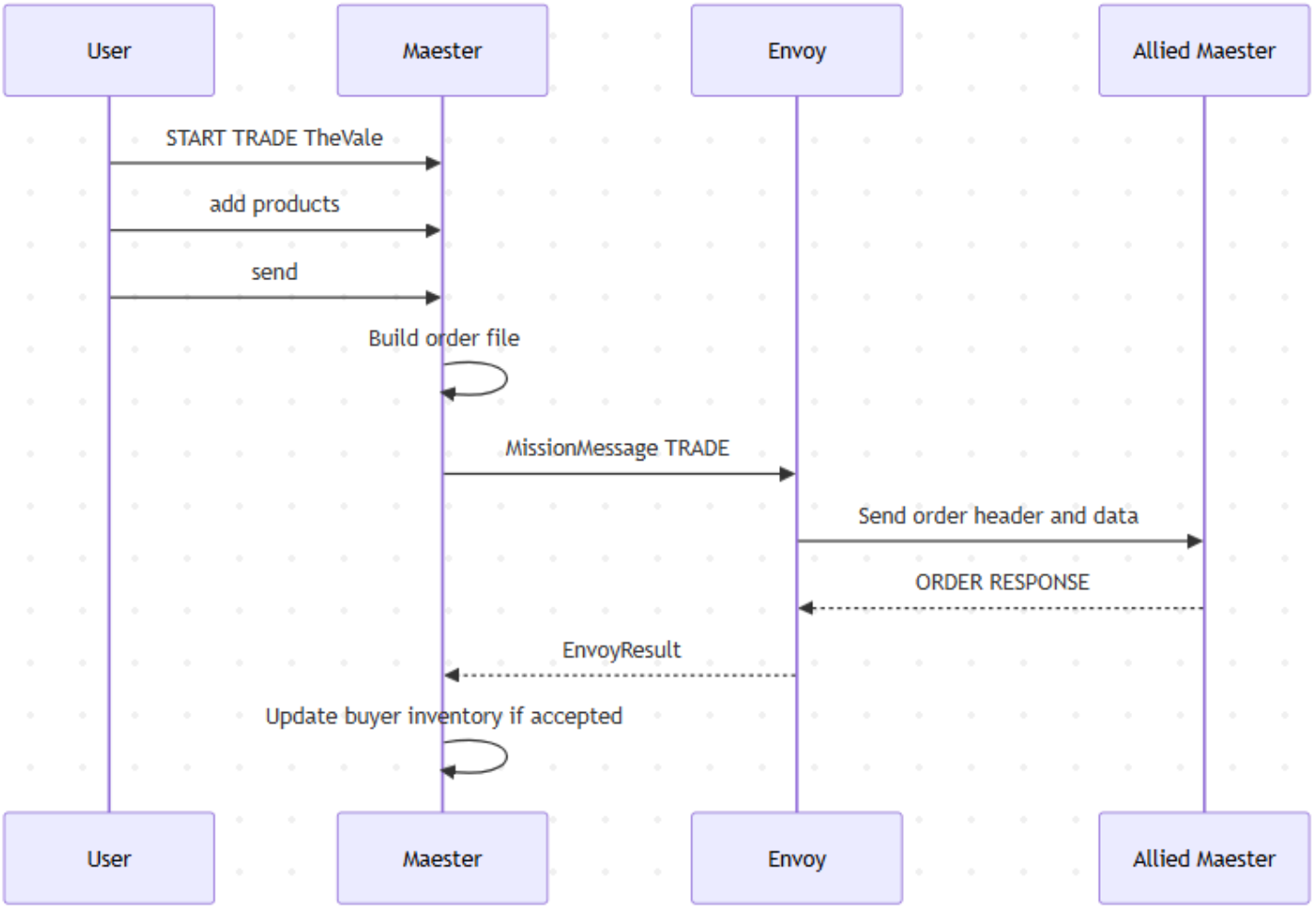
    build_order_file(maester, realm, items, item_count,
order_path);

    memset(&mission, 0, sizeof(MissionMessage));
    mission.type = MISSION_TRADE;
    strncpy(mission.realm, realm, sizeof(mission.realm) - 1);
    strncpy(mission.order_path, order_path,
sizeof(mission.order_path) - 1);

    assign_envoy_mission(maester, envoy_id, &mission);
}
```

The shopping list is still assembled by the Maester because it depends on the current trade session and on the commands the user types, like add, remove, send , or cancel. After the user decides to send, the Maester writes the order file and then assigns the network transfer to an Envoy.

The Envoy then handles the remote side i mean of the operation, by connecting to the destination Maester, pushing the order header along with the file data, then waiting for the response , and finally reporting the result back. In this way the Maester stays responsive while the trade request is still being processed.


User
Maester
Envoy
Allied Maester
START TRADE TheVale
add products
send
Build order file
MissionMessage TRADE
Send order header and data
ORDER RESPONSE
EnvoyResult
Update buyer inventory if accepted
User
Maester
Envoy
Allied Maester

Graceful Shutdown with Envoys

The cleanup process must also handle Envoy processes and their pipes. When the Maester exits, it stops accepting new missions, closes the communication pipes and waits for or terminates child processes. This is necessary because Envoys are real child processes, and leaving them alive after the Maester finishes would create orphan or zombie processes.

```
void cleanup_envoys(Maester *maester) {
    int i;

    i = 0;
    while (i < maester->num_envoys) {
        close(maester->envoys[i].to_envoy_fd);
        close(maester->envoys[i].from_envoy_fd);

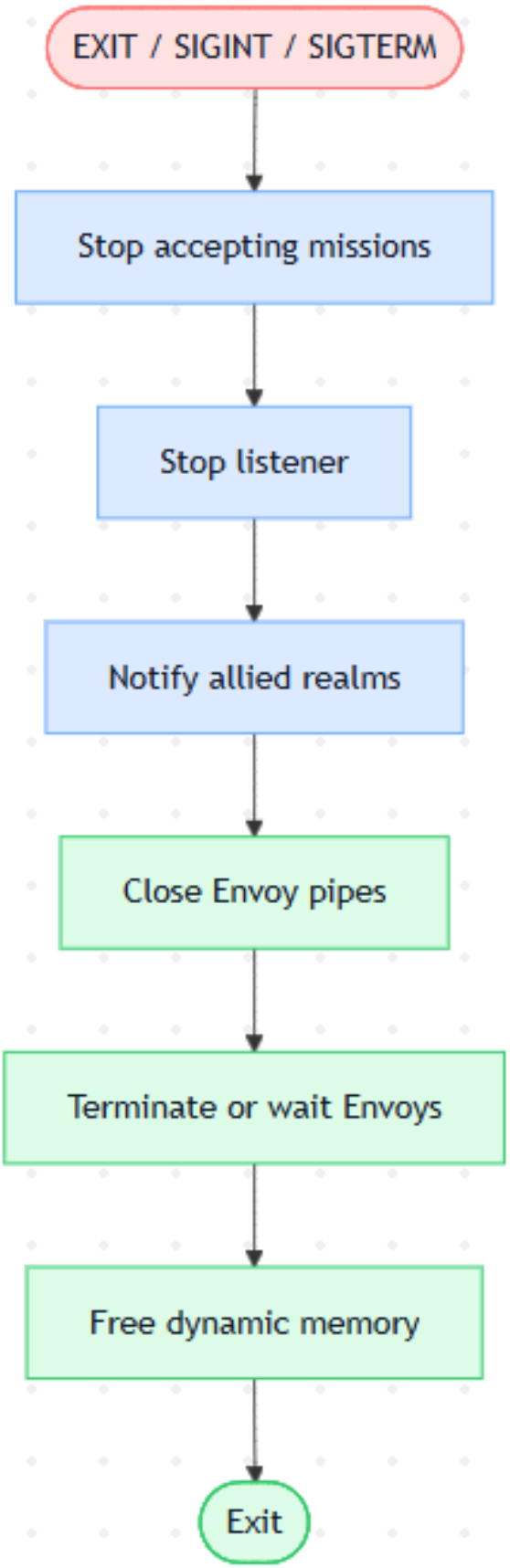
        if (maester->envoys[i].pid > 0) {
            kill(maester->envoys[i].pid, SIGTERM);
            waitpid(maester->envoys[i].pid, NULL, 0);
        }

        free(maester->envoys[i].mission_realm);
        i++;
    }

    free(maester->envoys);
    maester->envoys = NULL;
}
```

Also closing the pipe descriptors helps install Envoys that might be waiting for a new mission. After that, the Maester sends a termination signal to each child process and gathers the status using waitpid() . That prevents dangling child processes or open file descriptors once the Maester is done.

This shutdown part is tied into the general robustness of the system . Since the project can be tested with ordinary exits as well as interrupted executions, the Maester must always try to release its internal resources before it terminates.


EXIT / SIGINT / SIGTERM
Stop accepting missions
Stop listener
Notify allied realms
Close Envoy pipes
Terminate or wait Envoys
Free dynamic memory
Exit

Synchronization of Shared State

The command loop, the listener thread, and the Envoy result handler all touch the Maester state at the same time. Because of that, the critical sections are guarded with the Maester mutex. Otherwise, two execution paths could update the same structure simultaneously, and the system might slip into an inconsistent condition.

```
pthread_mutex_lock(&maester->lock);

/* modify alliances, inventory, pending pledges or envoy
status */

pthread_mutex_unlock(&maester->lock);
```

The most important design decision is that Envoys do not directly own the final state. They return results, and the Maester applies the final modifications. This keeps the official state centralized and prevents different processes from modifying the same structures independently.

Shared resources include:

```
alliances
pending pledges
inventory
remote product cache
Envoy status table
```

This sync model keeps the implementation more straightforward and safer as the number of concurrent missions goes up. The Maester becomes the single controlling authority for the realm state , while Envoys only carry out missions and then relay what happened.